

---

# Rapport Technique

## Service de Gestion des Véhicules

Microservice Spring Boot — CRUD, Kafka, GraphQL, Tests et  
Observabilité

---

Système de Gestion de Flotte de Véhicules

Master 1 GIL — 2025/2026

### Informations du projet

|                               |                                   |
|-------------------------------|-----------------------------------|
| <b>Auteurs :</b>              | Kherbach Rima & Mouhoubi Leiticia |
| <b>Formation :</b>            | Master 1 GIL                      |
| <b>Université :</b>           | Université de Rouen Normandie     |
| <b>Encadrants :</b>           | Lydia et Luc                      |
| <b>Année :</b>                | 2025 – 2026                       |
| <b>Semaines documentées :</b> | 1 et 2                            |

Mars 2026

# Chapitre 1

## Introduction

Ce rapport décrit en détail l'implémentation du microservice **vehicule-service**, un composant central du système de gestion de flotte de véhicules. Ce service gère le cycle de vie complet des véhicules : création, consultation, mise à jour et suppression (CRUD), tout en intégrant des technologies modernes pour la communication asynchrone (Kafka), les API flexibles (GraphQL), l'observabilité (OpenTelemetry) et les tests automatisés.

### 1.1 Architecture générale

Le service suit une architecture en couches classique :

1. **Couche Contrôleur** — Expose les API REST et GraphQL
2. **Couche Service** — Contient la logique métier
3. **Couche Repository** — Gère la persistance avec Spring Data JPA
4. **Couche Événementielle** — Publie et consomme des événements via Kafka

**Port du service** : 4000

**Base de données** : PostgreSQL (fleet\_vehicles)

**Messagerie** : Apache Kafka (localhost :9092)

**Tracing** : OpenTelemetry (OTLP sur localhost :4317)

### 1.2 Représentations graphiques de l'architecture

Pour mieux visualiser la structure logicielle et les flux de communication au sein du microservice, les deux diagrammes suivants détaillent l'implémentation de notre solution.

#### 1.2.1 Flux de communication et Composants

Le diagramme ci-dessous illustre le flux des requêtes depuis les clients (via les API REST ou GraphQL) jusqu'à la base de données PostgreSQL et le bus de messages Kafka. Il met en évidence le rôle central de la couche Service (**VehicleServiceImpl**) qui orchestre la logique métier, interagit avec le **Repository** pour la persistance, et délègue l'envoi d'événements asynchrones au **Producer** Kafka.

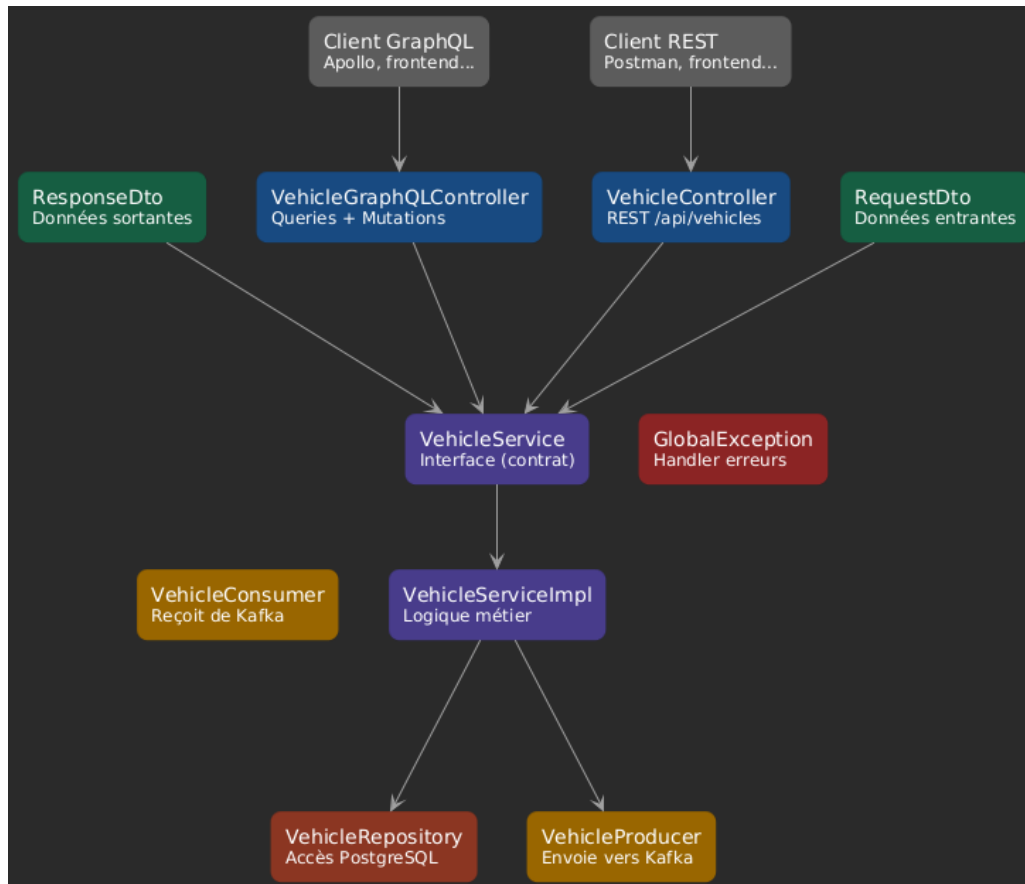


FIGURE 1.1 – Architecture détaillée et flux de données du microservice Véhicules

### 1.2.2 Diagramme de classes complet

La figure suivante présente l'organisation exhaustive des classes et interfaces du microservice, réparties par packages (**controllers**, **services**, **repository**, **entity**, **dto**, **kafka**, **exception**). Ce diagramme souligne l'utilisation des DTOs pour isoler l'entité JPA **Vehicle** des interfaces externes, garantissant ainsi un couplage faible et une meilleure maintenabilité.



# Chapitre 2

## Interface REST et Alignement OpenAPI

Le service expose une interface REST alignée sur la spécification OpenAPI de la passerelle (`gateway/openapi/vehicule-openapi.yaml`). Les points d'entrée utilisent le nommage en français (`/api/vehicules`) et le champ identifiant est `id_vehicule`.

### 2.1 Points d'entrée (Endpoints)

| Endpoints REST — Base path : <code>/api/vehicules</code> |                                  |                                            |
|----------------------------------------------------------|----------------------------------|--------------------------------------------|
| Méthode                                                  | Endpoint                         | Description                                |
| POST                                                     | <code>/</code>                   | Création d'un véhicule                     |
| GET                                                      | <code>/</code>                   | Liste de tous les véhicules                |
| GET                                                      | <code>/disponibles</code>        | Liste uniquement les véhicules disponibles |
| GET                                                      | <code>/{"id"}</code>             | Détail d'un véhicule par UUID              |
| PUT                                                      | <code>/{"id"}</code>             | Mise à jour complète                       |
| PATCH                                                    | <code>/{"id"}/statut</code>      | Mise à jour du statut (Kafka)              |
| PATCH                                                    | <code>/{"id"}/kilometrage</code> | Mise à jour du kilométrage                 |
| DELETE                                                   | <code>/{"id"}</code>             | Suppression du véhicule                    |

### 2.2 Structure des données (DTO)

Les réponses utilisent la structure suivante pour garantir la compatibilité avec le schéma SQL et la passerelle :

```
1 {
2   "id_vehicule": "550e8400-e29b-41d4-a716-446655440000",
3   "immatriculation": "AB-123-CD",
4   "marque": "Renault",
5   "modele": "Clio",
6   "annee": 2022,
7   "statut": "DISPONIBLE",
8   "kilometrage": 45000,
9   "type": "Berline"
10 }
```

---

Listing 2.1 – Exemple de réponse VehicleResponseDto

## 2.3 Ports et Connectivité

Le microservice est configuré pour s'intégrer dans un environnement distribué :

- **Port Interne (Service)** : 4000 (configuré dans `application.yml`)
- **Port Externe (Gateway)** : 3001 (port exposé par la passerelle pour les clients externes)

# Chapitre 3

## Bibliothèques et dépendances

Le projet utilise **Spring Boot 3.1.5** comme framework principal.

### 3.1 Dépendances principales

| Dépendances de production |                                   |                                          |
|---------------------------|-----------------------------------|------------------------------------------|
| Bibliothèque              | Artifact                          | Rôle                                     |
| Spring Boot Web           | spring-boot-starter-web           | Serveur HTTP embarqué (Tomcat), API REST |
| Spring Data JPA           | spring-boot-starter-data-jpa      | ORM Hibernate, mapping entité-table      |
| PostgreSQL Driver         | postgresql                        | Pilote JDBC pour PostgreSQL              |
| Validation                | spring-boot-starter-validation    | Validation des DTOs (@NotBlank, @Min)    |
| Spring Kafka              | spring-kafka                      | Producteur/consommateur Kafka            |
| Spring GraphQL            | spring-boot-starter-graphql       | API GraphQL avec schéma SDL              |
| Actuator                  | spring-boot-starter-actuator      | Endpoints de santé et métriques          |
| Micrometer                | micrometer-registry-prometheus    | Export des métriques vers Prometheus     |
| OpenTelemetry             | opentelemetry-spring-boot-starter | Tracing distribué                        |

## 3.2 Dépendances de test

| Dépendances de test |                          |                                           |
|---------------------|--------------------------|-------------------------------------------|
| Bibliothèque        | Artifact                 | Rôle                                      |
| Spring Boot Test    | spring-boot-starter-test | JUnit 5, Mockito, MockMvc                 |
| H2 Database         | h2                       | Base de données en mémoire pour les tests |
| Spring Kafka Test   | spring-kafka-test        | Kafka embarqué pour les tests             |
| Spring GraphQL Test | spring-graphql-test      | Outils de test GraphQL                    |



# Chapitre 4

## Modèle de données

### 4.1 Schéma SQL (PostgreSQL)

La table `vehicule` est définie avec les contraintes suivantes :

```
1 CREATE TYPE statut_vehicule AS ENUM (  
2     'DISPONIBLE', 'EN_COURSE', 'EN_MAINTENANCE'  
3 );  
4  
5 CREATE TABLE vehicule (  
6     id_vehicule      UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
7     immatriculation  VARCHAR(20)      NOT NULL UNIQUE,  
8     marque           VARCHAR(100)     NOT NULL,  
9     modele           VARCHAR(100)     NOT NULL,  
10    annee             SMALLINT         NOT NULL  
11                    CHECK (annee >= 1900  
12                    AND annee <= EXTRACT(YEAR FROM NOW()) + 1),  
13    statut            statut_vehicule NOT NULL DEFAULT 'DISPONIBLE',  
14    kilometrage       INTEGER          NOT NULL DEFAULT 0  
15                    CHECK (kilometrage >= 0),  
16    type              VARCHAR(50)      NOT NULL,  
17    created_at        TIMESTAMPTZ      NOT NULL DEFAULT NOW(),  
18    updated_at        TIMESTAMPTZ  
19 );
```

Listing 4.1 – Script SQL de création de la table `vehicule`

#### Points clés du schéma :

- **UUID** comme clé primaire, généré via `gen_random_uuid()`
- **Enum PostgreSQL** pour le statut, garantissant l'intégrité
- **Contraintes CHECK** sur l'année et le kilométrage
- **Unicité** de l'immatriculation
- **Index** sur statut et immatriculation

## 4.2 Entité JPA

L'entité Java `Vehicle` est mappée sur la table SQL :

```
1 @Entity
2 @Table(name = "vehicule")
3 public class Vehicle {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.UUID)
7     @Column(name = "id_vehicule")
8     private UUID id;
9
10    @Column(name = "immatriculation", nullable = false,
11            unique = true, length = 20)
12    private String immatriculation;
13
14    @Column(nullable = false, length = 100)
15    private String marque;
16
17    @Column(nullable = false, length = 100)
18    private String modele;
19
20    @Column(name = "annee", nullable = false)
21    private Short annee;
22
23    @Column(nullable = false)
24    @Enumerated(EnumType.STRING)
25    private VehicleStatus statut = VehicleStatus.DISPONIBLE;
26
27    @Column(nullable = false)
28    private Integer kilometrage = 0;
29
30    public enum VehicleStatus {
31        DISPONIBLE, EN_COURSE, EN_MAINTENANCE
32    }
33
34    @PrePersist
35    protected void onCreate() {
36        this.createdAt = LocalDateTime.now();
37        this.updatedAt = LocalDateTime.now();
38    }
39
40    @PreUpdate
41    protected void onUpdate() {
42        this.updatedAt = LocalDateTime.now();
43    }
44 }
```

Listing 4.2 – Extrait de `Vehicle.java`

## 4.3 Correspondance entité / table

### Mapping JPA — PostgreSQL

| Champ Java      | Colonne SQL     | Type SQL        |
|-----------------|-----------------|-----------------|
| id (UUID)       | id_vehicule     | UUID            |
| immatriculation | immatriculation | VARCHAR(20)     |
| marque          | marque          | VARCHAR(100)    |
| modele          | modele          | VARCHAR(100)    |
| annee (Short)   | annee           | SMALLINT        |
| statut (Enum)   | statut          | statut_vehicule |
| kilometrage     | kilometrage     | INTEGER         |
| type            | type            | VARCHAR(50)     |
| createdAt       | created_at      | TIMESTAMPTZ     |
| updatedAt       | updated_at      | TIMESTAMPTZ     |

# Chapitre 5

## API REST — CRUD

Le contrôleur REST expose les opérations CRUD classiques via `VehicleController`.

### 5.1 Endpoints disponibles

Endpoints REST du service véhicule

| Méthode | URL                      | Action   | Description                  |
|---------|--------------------------|----------|------------------------------|
| POST    | /api/vehicles            | Create   | Créer un nouveau véhicule    |
| GET     | /api/vehicles            | Read All | Lister tous les véhicules    |
| GET     | /api/vehicles/{id}       | Read One | Obtenir un véhicule par UUID |
| GET     | /api/vehicles/statut/{s} | Filter   | Filtrer par statut           |
| PUT     | /api/vehicles/{id}       | Update   | Mettre à jour un véhicule    |
| DELETE  | /api/vehicles/{id}       | Delete   | Supprimer un véhicule        |

### 5.2 Contrôleur REST

```
1 @RestController
2 @RequestMapping("/api/vehicles")
3 public class VehicleController {
4
5     private final VehicleService vehicleService;
6
7     @PostMapping
8     public ResponseEntity<VehicleResponseDto> createVehicle(
9         @Valid @RequestBody VehicleRequestDto request) {
10         VehicleResponseDto response =
11             vehicleService.createVehicle(request);
12         return ResponseEntity.status(HttpStatus.CREATED)
13             .body(response);
14     }
15 }
```

```
14     }
15
16     @GetMapping("/{id}")
17     public ResponseEntity<VehicleResponseDto> getVehicleById(
18         @PathVariable UUID id) {
19         VehicleResponseDto vehicle =
20             vehicleService.getVehicleById(id);
21         return ResponseEntity.ok(vehicle);
22     }
23
24     @PutMapping("/{id}")
25     public ResponseEntity<VehicleResponseDto> updateVehicle(
26         @PathVariable UUID id,
27         @Valid @RequestBody VehicleRequestDto request) {
28         VehicleResponseDto response =
29             vehicleService.updateVehicle(id, request);
30         return ResponseEntity.ok(response);
31     }
32
33     @DeleteMapping("/{id}")
34     public ResponseEntity<Void> deleteVehicle(
35         @PathVariable UUID id) {
36         vehicleService.deleteVehicle(id);
37         return ResponseEntity.noContent().build();
38     }
39 }
```

Listing 5.1 – VehicleController.java

## 5.3 DTOs (Data Transfer Objects)

Le projet utilise deux DTOs pour découpler l'API de l'entité interne :

- **VehicleRequestDto** — Données entrantes avec validation :
  - `@NotBlank` sur marque, modèle, immatriculation, type
  - `@NotNull` sur l'année
  - `@Min(0)` sur le kilométrage
- **VehicleResponseDto** — Données sortantes avec ID UUID, timestamps, etc.

## 5.4 Couche Service

```
1 @Service
2 @Transactional
3 public class VehicleServiceImpl implements VehicleService {
4
5     private final VehicleRepository vehicleRepository;
6     private final VehicleProducer vehicleProducer;
7 }
```

```
8      @Override
9      public VehicleResponseDto createVehicle(
10          VehicleRequestDto request) {
11          // Verification d'unicite de l'immatriculation
12          if (vehicleRepository.existsByImmatriculation(
13              request.getImmatriculation())) {
14              throw new IllegalArgumentException(
15                  "Immatriculation deja existante");
16          }
17          Vehicle vehicle = mapToEntity(request);
18          Vehicle saved = vehicleRepository.save(vehicle);
19
20          // Publication de l'evenement Kafka
21          vehicleProducer.sendVehicleEvent(
22              "VEHICLE_CREATED", mapToDto(saved));
23          return mapToDto(saved);
24      }
25 }
```

Listing 5.2 – Extrait de VehicleServiceImpl.java

**Points importants de la couche Service :**

- `@Transactional` assure l'intégrité des opérations
- `@Transactional(readOnly = true)` optimise les lectures
- Vérifications métier (unicité d'immatriculation)
- Publication automatique d'événements Kafka après chaque opération

# Chapitre 6

## Apache Kafka — Communication asynchrone

### 6.1 Rôle de Kafka dans l'architecture

Apache Kafka est utilisé comme système de messagerie asynchrone pour :

- **Découpler** les services — le producteur n'attend pas de réponse
- **Publier des événements** sur le cycle de vie des véhicules
- **Permettre le replay** des événements en cas de besoin
- **Alimenter d'autres services** (notifications, analytics, etc.)

### 6.2 Producteur Kafka

```
1 @Service
2 public class VehicleProducer {
3
4     private final KafkaTemplate<String, String> kafkaTemplate;
5     private final ObjectMapper objectMapper;
6
7     @Value("${app.kafka.topic.vehicle-events:vehicle-events}")
8     private String vehicleEventsTopic;
9
10    public void sendVehicleEvent(String eventType,
11                                VehicleResponseDto vehicle) {
12        String key = eventType + "-" + vehicle.getId();
13        String payload = objectMapper.writeValueAsString(
14            new VehicleEvent(eventType, vehicle));
15        kafkaTemplate.send(vehicleEventsTopic, key, payload);
16    }
17
18    // Classe interne -- structure de l'événement
19    public static class VehicleEvent {
20        private String eventType;
21        private VehicleResponseDto vehicle;
22        private long timestamp;
```

```

23     }
24 }

```

Listing 6.1 – VehicleProducer.java

### Types d'événements Kafka

| Événement       | Déclencheur               |
|-----------------|---------------------------|
| VEHICLE_CREATED | Création d'un véhicule    |
| VEHICLE_UPDATED | Mise à jour d'un véhicule |
| VEHICLE_DELETED | Suppression d'un véhicule |

## 6.3 Consommateur Kafka

```

1 @Service
2 public class VehicleConsumer {
3
4     @KafkaListener(
5         topics = "${app.kafka.topic.vehicle-events}",
6         groupId = "${spring.kafka.consumer.group-id}")
7     public void consumeVehicleEvent(String message) {
8         log.info("Evenement vehicule recu: {}", message);
9         // Traitement : notifications, mises a jour, etc.
10    }
11 }

```

Listing 6.2 – VehicleConsumer.java

## 6.4 Configuration Kafka

```

1 spring:
2   kafka:
3     bootstrap-servers: localhost:9092
4     producer:
5       key-serializer: org.apache.kafka.common.serialization
6                       .StringSerializer
7       value-serializer: org.apache.kafka.common.serialization
8                        .StringSerializer
9     consumer:
10      group-id: vehicule-service-group
11      auto-offset-reset: earliest

```

Listing 6.3 – Configuration Kafka dans application.yml



# Chapitre 7

## GraphQL — API flexible

### 7.1 Pourquoi GraphQL ?

GraphQL offre des avantages complémentaires à REST :

- **Requêtes flexibles** — Le client choisit exactement les champs voulus
- **Un seul endpoint** — `/graphql` remplace plusieurs endpoints REST
- **Pas de sur-récupération** — Contrairement à REST qui retourne tous les champs
- **Introspection** — Documentation automatique du schéma

### 7.2 Schéma GraphQL

```
1 type Vehicle {
2   id: ID!
3   marque: String!
4   modele: String!
5   immatriculation: String!
6   statut: String!
7   kilometrage: Int
8   type: String!
9   annee: Int!
10  createdAt: String
11  updatedAt: String
12 }
13
14 type Query {
15   allVehicles: [Vehicle!]!
16   vehicleById(id: ID!): Vehicle
17   vehiclesByStatut(statut: String!): [Vehicle!]!
18 }
19
20 type Mutation {
21   createVehicle(
22     marque: String!, modele: String!,
23     immatriculation: String!, type: String!,
24     statut: String, kilometrage: Int, annee: Int!
```

```
25     ): Vehicle!  
26     updateVehicle(  
27         id: ID!, marque: String!, modele: String!,  
28         immatriculation: String!, type: String!,  
29         statut: String, kilometrage: Int, annee: Int!  
30     ): Vehicle!  
31     deleteVehicle(id: ID!): Boolean!  
32 }
```

Listing 7.1 – schema.graphqls

## 7.3 Contrôleur GraphQL

```
1 @Controller  
2 public class VehicleGraphQLController {  
3  
4     private final VehicleService vehicleService;  
5  
6     @QueryMapping  
7     public List<VehicleResponseDto> allVehicles() {  
8         return vehicleService.getAllVehicles();  
9     }  
10  
11     @QueryMapping  
12     public VehicleResponseDto vehicleById(@Argument String id) {  
13         return vehicleService.getVehicleById(  
14             UUID.fromString(id));  
15     }  
16  
17     @MutationMapping  
18     public VehicleResponseDto createVehicle(  
19         @Argument String marque,  
20         @Argument String modele,  
21         @Argument String immatriculation,  
22         @Argument String type,  
23         @Argument Integer annee) {  
24         VehicleRequestDto request = new VehicleRequestDto();  
25         request.setMarque(marque);  
26         request.setModele(modele);  
27         request.setImmatriculation(immatriculation);  
28         request.setType(type);  
29         request.setAnnee(annee != null  
30             ? annee.shortValue() : null);  
31         return vehicleService.createVehicle(request);  
32     }  
33 }
```

Listing 7.2 – Extrait de VehicleGraphQLController.java

## 7.4 Exemple de requête GraphQL

```
1 query {  
2   allVehicles {  
3     marque  
4     modele  
5     statut  
6   }  
7 }
```

Listing 7.3 – Requête pour obtenir marque et modèle

L'interface **GraphiQL** est activée et accessible à <http://localhost:4000/graphiql> pour tester les requêtes de manière interactive.

# Chapitre 8

## Tests automatisés

### 8.1 Stratégie de test

Le projet implémente deux niveaux de tests :

1. **Tests unitaires** — Couche service avec Mockito
2. **Tests d'intégration** — Couche contrôleur avec MockMvc

### 8.2 Tests unitaires de la couche Service

Les tests de `VehicleServiceTest` utilisent **Mockito** pour isoler la logique métier :

```
1 @ExtendWith(MockitoExtension.class)
2 class VehicleServiceTest {
3
4     @Mock
5     private VehicleRepository vehicleRepository;
6     @Mock
7     private VehicleProducer vehicleProducer;
8     @InjectMocks
9     private VehicleServiceImpl vehicleService;
10
11     @Test
12     void createVehicle_Success() {
13         when(vehicleRepository.existsByImmatriculation(
14             "AB-123-CD")).thenReturn(false);
15         when(vehicleRepository.save(any(Vehicle.class)))
16             .thenReturn(vehicle);
17
18         VehicleResponseDto result =
19             vehicleService.createVehicle(requestDto);
20
21         assertNotNull(result);
22         assertEquals("Renault", result.getMarque());
23         verify(vehicleProducer).sendVehicleEvent(
24             eq("VEHICLE_CREATED"),
25             any(VehicleResponseDto.class));
26     }
27 }
```

```
26     }
27
28     @Test
29     void createVehicle_DuplicateImmatriculation() {
30         when(vehicleRepository.existsByImmatriculation(
31             "AB-123-CD")).thenReturn(true);
32
33         assertThrows(IllegalArgumentException.class,
34             () -> vehicleService.createVehicle(requestDto));
35         verify(vehicleRepository, never())
36             .save(any(Vehicle.class));
37     }
38 }
```

Listing 8.1 – Extrait de VehicleServiceTest.java

## 8.3 Tests d'intégration du contrôleur REST

```
1 @WebMvcTest(VehicleController.class)
2 class VehicleControllerTest {
3
4     @Autowired
5     private MockMvc mockMvc;
6     @MockBean
7     private VehicleService vehicleService;
8
9     @Test
10    void createVehicle_ReturnsCreated() throws Exception {
11        when(vehicleService.createVehicle(
12            any(VehicleRequestDto.class)))
13            .thenReturn(responseDto);
14
15        mockMvc.perform(post("/api/vehicles")
16            .contentType(MediaType.APPLICATION_JSON)
17            .content(objectMapper
18                .writeValueAsString(requestDto)))
19            .andExpect(status().isCreated())
20            .andExpect(jsonPath("$.marque").value("Renault"))
21            .andExpect(jsonPath("$.immatriculation")
22                .value("AB-123-CD"));
23    }
24
25    @Test
26    void createVehicle_ValidationError() throws Exception {
27        VehicleRequestDto invalid = new VehicleRequestDto();
28
29        mockMvc.perform(post("/api/vehicles")
30            .contentType(MediaType.APPLICATION_JSON)
31            .content(objectMapper
```

```

32         .writeValueAsString(invalid)))
33         .andExpect(status().isBadRequest());
34     }
35 }

```

Listing 8.2 – Extrait de VehicleControllerTest.java

## 8.4 Cas de test couverts

### Couverture des tests

| Classe     | Test                    | Scénario                           |
|------------|-------------------------|------------------------------------|
| Service    | createVehicle_Success   | Création réussie + événement Kafka |
| Service    | createVehicle_Duplicate | Rejet si immatriculation dupliquée |
| Service    | getVehicleById_Success  | Récupération par UUID              |
| Service    | getVehicleById_NotFound | Exception si inexistant            |
| Service    | getAllVehicles_Success  | Liste complète                     |
| Service    | getVehiclesByStatut     | Filtrage par statut                |
| Service    | updateVehicle_Success   | Mise à jour + événement Kafka      |
| Service    | updateVehicle_NotFound  | Exception si inexistant            |
| Service    | deleteVehicle_Success   | Suppression + événement Kafka      |
| Service    | deleteVehicle_NotFound  | Exception si inexistant            |
| Controller | createVehicle_Created   | HTTP 201 + JSON                    |
| Controller | getAllVehicles_List     | HTTP 200 + tableau                 |
| Controller | getVehicleById          | HTTP 200 + détail                  |
| Controller | updateVehicle_Updated   | HTTP 200 + modifié                 |
| Controller | deleteVehicle_NoContent | HTTP 204                           |
| Controller | validation_BadRequest   | HTTP 400 si invalide               |

# Chapitre 9

## Observabilité — OpenTelemetry SDK

L'implémentation de l'observabilité repose sur les **3 piliers d'OpenTelemetry** (Traces, Métriques, Logs), intégrés via le SDK Java.

### 9.1 Le SDK et la configuration centralisée

Une classe `VehicleTelemetry` centralise l'accès au **Tracer** (pour les traces) et au **Meter** (pour les métriques).

```
1 @Component
2 public class VehicleTelemetry {
3     private final Tracer tracer;
4     private final LongCounter vehiclesCreatedCounter;
5     private final LongHistogram operationDurationHistogram;
6
7     public VehicleTelemetry(OpenTelemetry openTelemetry) {
8         this.tracer = openTelemetry.getTracer("vehicule-service");
9         Meter meter = openTelemetry.getMeter("vehicule-service");
10
11         // Compteur : nombre total de creations
12         this.vehiclesCreatedCounter = meter.counterBuilder("
13             vehicles.created.total").build();
14
15         // Histogramme : duree des operations
16         this.operationDurationHistogram = meter.histogramBuilder("
17             vehicles.operation.duration").ofLongs().build();
18     }
19 }
```

Listing 9.1 – Extraits de `VehicleTelemetry.java`

### 9.2 Pilier 1 : Traces Distribuées

Les traces permettent de suivre une requête. Nous utilisons deux méthodes :

- **Instrumentation déclarative** : Utilisation de `@WithSpan` pour créer automatiquement un span sur une méthode.

- **Attributs personnalisés** : Utilisation de `@SpanAttribute` pour injecter des paramètres (ex : UUID du véhicule) dans la trace.

```
1 @WithSpan("createVehicle")
2 public VehicleResponseDto createVehicle(VehicleRequestDto request)
3 {
4     Span.current().setAttribute("vehicle.marque", request.getMarque());
5     // ... logique metier ...
6 }
```

Listing 9.2 – Traces dans la couche Service

## 9.3 Pilier 2 : Métriques Personnalisées

Contrairement aux traces, les métriques sont des données numériques agrégées. Nous mesurons :

- Le nombre total de véhicules créés et supprimés.
- Le taux d’erreurs par type d’opération.
- La distribution des temps de traitement (latence).

## 9.4 Pilier 3 : Logs Structurés et Corrélation

Pour relier les logs aux traces, nous injectons le `traceId` et le `spanId` dans le contexte de diagnostic (**MDC**).

```
1 <encoder>
2     <pattern>
3         %d %-5level [traceId=%X{traceId:-none}, spanId=%X{spanId:-none}] %logger - %msg%n
4     </pattern>
5 </encoder>
```

Listing 9.3 – Configuration Logback (logback-spring.xml)

## 9.5 Vérification du fonctionnement

Pour s’assurer que l’observabilité est active :



### Plan de vérification

1. **Logs** : Lancer l'application et effectuer un appel API. Le log doit afficher `traceId=...` avec un ID hexadécimal au lieu de `none`.
2. **Métriques** : Accéder à `/actuator/prometheus`. Rechercher la ligne `vehicles_created_total` pour voir le compteur augmenter.
3. **Traces** : Si un collecteur (Jaeger/Tempo) est configuré, les spans `createVehicle` apparaîtront avec leurs attributs (`vehicle.marque`).

# Chapitre 10

## Configuration de l'application

```
1 server:
2   port: 4000
3
4 spring:
5   application:
6     name: vehicule-service
7   datasource:
8     url: jdbc:postgresql://localhost:5432/fleet_vehicles
9     username: postgres
10    password: postgres
11  jpa:
12    hibernate:
13      ddl-auto: update
14    properties:
15      hibernate:
16        dialect: org.hibernate.dialect.PostgreSQLDialect
17  graphql:
18    graphiql:
19      enabled: true
20    path: /graphql
21
22 management:
23   endpoints:
24     web:
25       exposure:
26         include: health,info,metrics,prometheus
```

Listing 10.1 – application.yml – Configuration complète

**Note :** Le mode `ddl-auto: update` permet à Hibernate de mettre à jour le schéma automatiquement au démarrage. En production, il est recommandé d'utiliser un outil de migration comme **Flyway** ou **Liquibase**.

# Chapitre 11

## Conclusion

Le microservice **vehicule-service** implémente une architecture moderne et complète :

- **CRUD REST** structuré avec validation, DTOs et gestion d’erreurs
- **GraphQL** offrant une API flexible complémentaire
- **Kafka** pour la communication événementielle asynchrone inter-services
- **Tests automatisés** couvrant la logique métier et les endpoints HTTP
- **Observabilité** via OpenTelemetry, Actuator et Prometheus
- **Cohérence entité/base** avec un mapping JPA aligné sur le schéma PostgreSQL

Cette architecture permet au service d’être *scalable*, *maintenable* et *observable* dans un contexte de déploiement microservices.